

Optimisation : méthode de la plus forte pente**Exercice 11**

Soit f la fonction de $\mathbf{R}^2$ dans $\mathbf{R}$ définie par

$$f(x_1, x_2) = \frac{1}{2}x_1^2 + 2x_2^2,$$

et le point $X_0 = (1, 1)$. On considère les trois directions suivantes

$$d_1 = -\nabla f(X_0), \quad d_2 = (-1, -1), \quad d_3 = (1, -3).$$

1. Vérifier que ces trois directions sont des directions de descente de f en X_0 .
2. Faire sur une même figure en Matlab le tracé de $f(X_0 + \alpha d)$ pour chacune de ces directions, avec $\alpha \in [0, 1]$.
3. Constaté que d_1 est bien la direction de plus forte descente. Quelle autre remarque peut-on faire ?

Exercice 12 : mise en place d'un code d'optimisation.

L'objectif de cet exercice est de comprendre l'articulation entre un code d'optimisation et le simulateur (la fonction à minimiser). On commence par un exemple très simple où l'on peut calculer explicitement le pas optimal.

On s'intéresse donc au problème

$$\min_x f(x). \tag{1}$$

1. Simulateur : on considère la fonction définie par

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2.$$

Ecrire dans un fichier `simf0.m` une fonction Matlab se présentant de la façon suivante :

`function [f,g]=simf0(x)`

En entrée, `x` est la valeur de la variable où il faut évaluer `f` et `g`,

En sortie, `f` vaudra $f_0(x)$, et `g` vaudra $\nabla f_0(x)$ la valeur du gradient de f_0 au point `x`.

2. Optimiseur

- (a) Calculer à la main la solution du problème (1) avec la fonction $f = f_0$ donnée.
- (b) Programmer dans un fichier `pfd.m` l'algorithme de plus forte descente vu en cours : calcul de la direction de plus forte descente ; calcul du pas optimal α ; définition du nouvel itéré. Le critère d'arrêt est le suivant : l'optimiseur considérera que l'optimum est atteint si la norme des conditions d'optimalité a été multiplié par `epsrel`. Il s'arrêtera donc si

$$\|\nabla f(x_k)\|_2 \leq \text{epsrel} \|\nabla f(x_0)\|_2.$$

On prendra pour point de départ $x_0 = (9, 1)$ et la valeur 10^{-8} pour `epsrel`.

3. Tests numériques

- (a) Afin d'illustrer la lenteur de cette méthode, faire afficher le nombre d'itérations nécessaires à la convergence de cet algorithme, pour cette fonction f_0 très simple.
- (b) Programmer dans le fichier `pfd.m` la tracé des itérés successifs ainsi que les courbes de niveau de la fonction f_0 . On pourra utiliser l'instruction `ezcontour` ou `contour`.
- (c) Programmer également le tracé de $f(x_k)$ et de $\|\nabla f(x_k)\|$ en fonction de k , en échelle semi-logarithmique.

Optimisation : recherche linéaire**Exercice 13 : première condition de Wolfe (diminution suffisante)**

On considère de nouveau la fonction f_0 définie par

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2,$$

le point $x_0 = (10, 1)$, les deux directions $D_1 = (-10, -9)$ et $D_2 = (-2, 1)$. On considère les deux directions normalisées

$$d_1 = D_1/||D_1|| \quad \text{et} \quad d_2 = D_2/||D_2||.$$

Afin d'illustrer le rôle de β_1 dans la première condition de Wolfe, pour $\alpha \in [0, 5]$:

1. Pour $\beta_1 = 0.5$,
 - (a) Tracer en Matlab sur une même figure : la courbe $f_0(x_0 + \alpha d_1)$, la droite $f_0(x_0) + \alpha \beta_1 \nabla f_0(x_0)^T d_1$ et la droite $f_0(x_0) + \alpha \nabla f_0(x_0)^T d_1$.
 - (b) Même question avec d_2 .
2. Même question avec $\beta_1 = 0.1$.

Exercice 14 : deuxième condition de Wolfe (progrès suffisant)

On rappelle que la seconde condition de Wolfe s'écrit

$$\nabla f(x_k + \alpha_k d_k)^T d_k \geq \beta_2 \nabla f(x_k)^T d_k,$$

avec $0 \leq \beta_2 \leq 1$. Cette condition peut aussi s'écrire

$$\frac{\nabla f(x_k + \alpha_k d_k)^T d_k}{\nabla f(x_k)^T d_k} \leq \beta_2.$$

Nous allons reprendre la fonction f_0 définie précédemment, le point $x_0 = (10, 1)$, et les deux mêmes directions d_1 et d_2 . Afin d'illustrer la seconde condition de Wolfe, pour $\alpha \in [0, 5]$:

1. Tracer en Matlab la courbe $\frac{\nabla f_0(x_0 + \alpha d_1)^T d_1}{\nabla f_0(x_0)^T d_1}$. Pourquoi son graphe est-il une droite ?
2. Même question avec d_2 .
3. En prenant $\beta_2 = 0.5$ quels sont les pas acceptables selon la deuxième condition de Wolfe ? Et pour $\beta_2 = 0.1$? $\beta_2 = 0.9$?

Exercice 15 : programmation de la recherche linéaire de Wolfe

1. Programmer, dans un fichier `wolfe.m`, l'algorithme vu en cours de recherche linéaire dans une fonction du type

```
function [xnv,nbsim]=wolfe('sim',xk,alpha,dk,beta1,beta2)
```

dans laquelle

- 'sim' est le nom du simulateur,
- `xk` est l'itéré courant,
- `alpha` est la valeur initiale du pas,
- `dk` est la direction de descente courante,

- `beta1` et `beta2` sont les paramètres β_1 et β_2 de la recherche linéaire de Wolfe,
 - `xnv` est le nouveau point $x + \alpha d$ avec le α trouvé par l'algorithme de Wolfe,
 - et `nbsim` est le nombre d'appels au simulateur qui ont été effectués au cours de la recherche linéaire.
2. Dans un fichier `gradrl.m`, modifier le code écrit dans `pfd.m` (feuille de TP précédente) en y ajoutant une recherche linéaire, c'est-à-dire en faisant appel à la fonction `wolfe` tout juste écrite pour trouver le pas.
 3. Utiliser `gradrl.m` pour chercher le minimum de f_0 . Quel est le gain en terme du nombre d'itérations ? Du nombre d'appels au simulateur f_0 ?
 4. Même question pour chercher le minimum de la fonction f_1 définie par

$$f_1(x) = \frac{1}{2}x_1^2 + x_1 \cos(x_2).$$

Optimisation : comparaison de différentes méthodes de descente**Exercice 16 : implémentation de la méthode de Newton locale**

On rappelle que la méthode de Newton locale est une méthode itérative dont chaque pas est défini par l'itération suivante :

$$H(x_k)d = -\nabla f(x_k),$$

où $H(x_k)$ est la hessienne de f au point x_k . Cette équation a un sens seulement lorsque la hessienne $H(x_k)$ est définie positive. La méthode de Newton a une convergence quadratique sous certaines conditions favorables.

1. Modification du simulateur :

La méthode de Newton nécessite de connaître la hessienne. Construire une nouvelle version `simf0_2` du simulateur `simf0` qui retourne aussi la hessienne `h` de la fonction f_0 au point x . Cette fonction commencera par l'en-tête :

```
function [f,g,h]=simf0_2(x)
```

2. Implémentation de la méthode de Newton avec pas adaptatif :

Programmer l'algorithme de Newton local avec pas adaptatif (selon la méthode de Wolfe) dans un fichier `newton_loc.m`.

3. Tests numériques

- (a) Tester la méthode de Newton sur la fonction f_0 . Commenter le résultat.
- (b) Programmer un simulateur `simf1_2` correspondant à la fonction définie pour $x \in \mathbf{R}$ par

$$f_1(x) = \frac{1}{2}x_1^2 + x_1 \cos(x_2).$$

Déterminer (analytiquement) les points critiques de f_1 . Parmi ceux-ci, lesquels sont des minima de f_1 ?

- (c) Tester la méthode de Newton pour la minimisation de f_1 en partant d'un point proche du minimum, par exemple $(-0.9, 0.4)$. Observer la convergence quadratique asymptotique.
- (d) Essayer en prenant des points de départ plus éloignés, et observer le défaut de convergence globale.

Exercice 17 : comparaison de la méthode de gradient et de la méthode de Newton locale

Comparer, en prenant différents points de départ, les méthodes de gradient avec pas adaptatif et de Newton locale avec pas adaptatif pour trouver le minimum des fonctions suivantes :

$$f_2(x) = x_1^4 + x_2^4,$$

$$f_3(x) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2.$$

pour f_3 : point de départ autour de $(-1.5, -1.5)$

Pour chaque cas, on déterminera

- 1. une carte des courbes de niveau et les itérés successifs de chaque méthode,
- 2. l'historique de convergence : l'évolution avec les itérations de f et de $\|\nabla f\|$.

Optimisation : Méthode de Gauss-Newton

Exercice 18 : meilleure gaussienne qui passe par un nuage de points

On se donne une famille de n points du plan $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ et on cherche la courbe gaussienne qui approche le mieux ce nuage de points, au sens des moindres carrés. La fonction que l'on cherche est de la forme

$$g(x) = \alpha_1 + \alpha_2 \exp(-\alpha_4(x - \alpha_3)^2).$$

Cela revient à minimiser la fonction coût de 4 paramètres

$$f(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = \frac{1}{2} \sum_{i=1}^n (\alpha_1 + \alpha_2 \exp(-\alpha_4(x_i - \alpha_3)^2) - y_i)^2.$$

On peut écrire la fonction sous la forme

$$f(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = \frac{1}{2} \|h(\alpha_1, \alpha_2, \alpha_3, \alpha_4)\|^2,$$

où $h : \mathbf{R}^4 \rightarrow \mathbf{R}^n$ est une fonction qui a été détaillée en TD.

NB : attention ici pour avoir une courbe gaussienne le coefficient α_4 doit être strictement positif. Il faudrait en toute rigueur faire une optimisation sous contraintes, mais nous nous contenterons de vérifier que cette contrainte est satisfaite.

1. Ecrire une fonction Matlab `calculeh.m` qui prend en entrée un nuage de points, sous la forme d'une matrice à 2 lignes, la j -ième colonne étant $[x_j; y_j]$ et un point de $\mathbf{R}^4$ $a = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)$, et qui donne en sortie le vecteur $h(a)$.
2. Ecrire une fonction Matlab `calculeDh.m` qui prend en entrée un nuage de points, sous la forme d'une matrice à 2 lignes, la j -ième colonne étant $[x_j; y_j]$ et un point de $\mathbf{R}^4$ $a = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)$, et qui donne en sortie la matrice Jacobienne $Dh(a)$.
3. Dans un script `testDh.m`, valider le calcul de Dh en se plaçant en un point a , et en comparant la différence divisée $(h(a + \epsilon d) - h(a))/\epsilon$ avec $Dh(a).d$ pour différentes valeurs de ϵ .
4. Ecrire un script Matlab `interpogaussienne.m` dans lequel on définit un nuage de points, sous la forme d'une matrice à 2 lignes, la j -ième colonne étant $[x_j; y_j]$ et un point de départ $a^0 = (\alpha_1^0, \alpha_2^0, \alpha_3^0, \alpha_4^0)$. Le script `interpogaussienne.m` minimise la fonction f en utilisant l'algorithme de Gauss-Newton, et affiche dans une figure le nuage de points et la meilleure gaussienne estimée.
NB : la direction donnée par la méthode de Gauss-Newton est un peu trop grande. On fera une recherche linéaire, ou plus simplement on pourra se contenter de multiplier cette direction par 0.1.
5. Tester ce script sur différents exemples. Exemples suggérés (pour commencer) :

un exemple avec peu de points

```
X=[-1 -0.9 0 0.1 0.2 1 2 3 0.6 ; 0 0.1 0.2 0.5 0.49 0.45 0.1 0.1 0.6] ;
```

puis une gaussienne perturbée (on changera le niveau de bruit)

```
X1=[10*randn(1,100)]-1 ;
```

```
X=[X1 ; 2+3*exp(-0.05*X1.^ 2)+1*rand(1,100)] ;
```